

Marcar como hecha

11.4. Redireccionamiento y transferencia de peticiones

Parámetros session y más

Si se observan los ejemplos anteriores, queda claro que se puede llegar muy lejos en lo que se refiere a al desarrollo de sitios web dinámicos y aplicaciones web solo mediante usando documentos html y servlets, pero también está claro que para un sitio complejo puede ser extenso escribir un servlet y hacer que todo parezca real. Es algo que JSP y JSF tienen que resolver, pero antes de mostrar lo que es, te mostraré algunos ejemplos de servlets. No tanto por los servlets, sino más bien para poner algunos conceptos básicos sobre aplicaciones web.

Parámetros hacia servlets

Empezaré creando una nueva aplicación web que he llamado **Parametros**, y el resultado es de nuevo una aplicación con una página de inicio llamada index.html. Mostraré primero cómo transferir parámetros a un servlet cuando se inicializa. He añadido un servlet llamado **Init1**, y en la ventana de configuración he

he definido lo siguiente (fíjate en la marca de verificación de Añadir información al descriptor de despliegue (web.xml)):

Chequear Add information to deployment descriptor (web.xml)

Una vez creado el servlet correspondiente, NetBeans ha creado un documento XML en la carpeta WEB-INF:

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="3.1" xmlns="http://xmlns.jcp.org/xml/ns/javaee" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd">
    <servlet>
        <servlet-name>init1</servlet-name>
        <servlet-class>parametros.servlets.init1</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>init1</servlet-name>
        <url-pattern>/init1</url-pattern>
    </servlet-mapping>
    <session-config>
        <session-timeout>
            30
        </session-timeout>
    </session-config>
</web-app>
```

Es el archivo de configuración que se utiliza en relación con el despliegue de la aplicación. Debe tener en cuenta que el nombre de el servlet es Init1, como se selecciona en la ventana anterior y que es un nombre que hace referencia a la clase parameters.servlets.Init1Servlet. También debes notar que el servlet en la aplicación tiene la dirección (URL) /init1. A continuación he expandido el documento XML, como se muestra a continuación:

```
<servlet-class>servlets.init1</servlet-class>
    <init-param>
        <param-name>owner</param-name>
        <param-value>LaRosquilla data</param-value>
    </init-param>
</servlet>
```

Esto significa que he definido un par clave/valor con el nombre owner y el valor LaRosquilla data.

Ahora crearemos otro Servlet llamado Init2Servlet. En la creación

Si miras la ventana Configurar Despliegue de Servlets de NetBean arriba, la opción inferior es especificar parámetros de inicialización para un servlet.

Esto puede hacerse, en lugar de editar manualmente el documento XML, como se muestra arriba. Si añadimos otro servlet, esta vez llamado

Init2Servlet, he configurado el despliegue sin marcar la casilla Add information to deployment descriptor (web.xml)
Y abajo, en la ventana de creación, permite crear parámetros; name "owner" value "LaRosquilla data"

```
@WebServlet(name = "Init2Servlet", urlPatterns = {"/Init2Servlet"}, initParams = {
    @WebInitParam(name = "owner", value = "LaRosquilla data")})
```

Se utiliza durante el despliegue como una alternativa a web.xml y se puede ver cómo se definen el nombre y la URL. Por último, he añadido un parámetro. El código código para processRequest(), que se ve abajo y después añadido el segundo enlace en la página de inicio(ver debajo), el primero va a la página por defecto

Luego he cambiado el código de Init2Servlet por el siguiente, (sólo se muestra la primera parte de la clase):

```
protected void processRequest(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    response.setContentType("text/html;charset=UTF-8");
    try (PrintWriter out = response.getWriter()) {
        /* TODO output your page here. You may use following sample code. */
        out.println("<!DOCTYPE html>");
        out.println("<html>");
        out.println("<head>");
        out.println("<title>Servlet init1</title>");
        out.println("</head>");
        out.println("<body>");
        out.println("<h1>Saludos desde " + getServletConfig().getInitParameter("owner") + "</h1>");
        out.println("</body>");
        out.println("</html>");
    }
}
```

Aquí deberías notar que la clase Init1 no está decorada por un atributo @WebServlet. La razón es que el servlet en cuestión está definido esta vez en el fichero de configuración. El uso de anotaciones es una alternativa al fichero de configuración que facilita la escritura de un servlet (no es necesario mantener el fichero de configuración xml), pero a la inversa si, los cambios requieren que el código pueda ser traducido de nuevo mientras que el fichero de configuración puede ser mantenido sin necesidad de modificar el código del servlet. También hay que tener en cuenta que en el segundo, en processRequest() sólo he cambiado una línea (la antepenúltima sentencia) y que recupera el parámetro definido en el documento XML. Ahora con dos servlets, se puede hacer una petición al nuevo servlet y el A continuación, la página de inicio:

```
<html>
  <head>
    <title>Parametros</title>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
  </head>
  <body>
    <h1>Parámetros al servlet</h1>
    <p>hola</p>
    <p><a href="Init1">Init1</a></p>
    <p><a href="Init2Servlet">Init2</a></p>
  </body>
</html>
```

Así, las páginas y los enlaces nos muestran:

Parámetros al servlet

hola

Init1

Init2

el primer enlace:

?

Servlet Init1 at /Parametros

el segundo:

Saludos desde LaRosquilla data

Sesiones

Cuando se solicita una página web desde el navegador y el servidor responde con una respuesta, se hará como ya se ha mencionado de acuerdo con el protocolo HTTP. Este protocolo es básicamente apátrida, lo que significa que después de que el servidor haya contestado con la respuesta, el servidor se ha "olvidado" de la petición, y la próxima vez que se reciba una petición para la misma página, el servidor la percibirá como una nueva petición. Este modelo encaja mal con las aplicaciones web modernas, y se puede, entre otras cosas, resolver el problema utilizando el concepto de sesión.

Cuando un usuario abre el navegador y envía una petición a una nueva página, el servidor investigará si se incluye un identificador de sesión. No importa si se trata de una petición de una nueva página introducida en la barra de direcciones del navegador, y el servidor generará entonces un identificador de sesión y lo enviará con la respuesta de vuelta al cliente. Si a continuación el cliente realiza un envío o, por ejemplo, hace clic en un enlace que remite a una página de la misma aplicación web, el navegador enviará el ID de sesión, y el servidor lo incluirá en su respuesta. De esta forma, se ha introducido el término llamado sesión que existe mientras el servidor haga referencia a páginas dentro de la misma aplicación web. Esto se puede utilizar para definir datos que viven durante toda la sesión. Cuando el servidor crea un ID de sesión, también crea un objeto de sesión y este objeto contiene una colección para objetos Object. El servidor puede así crear y guardar objetos en esta colección, y estos objetos viven mientras viva el objeto de sesión, y por lo tanto pueden ser usados por todas las páginas dentro de esa sesión. Es importante tener en cuenta que los objetos sólo se conocen en el lado del servidor y no se envían al cliente. El único que se envía al cliente es el identificador de sesión, que se envía de un lado a otro para cada par petición/respuesta.

Los objetos de sesión individuales viven en el servidor hasta que se eliminan explícitamente o hasta que se eliminan debido a un tiempo de espera (por ejemplo, puede ser de 30 minutos). Esto significa, por otra parte, que el servidor puede tener muchos objetos de sesión que ya no se utilizan, con lo que ocupan espacio, y que en parte esos objetos de sesión (como un carrito de la compra) pueden desaparecer si se deja de trabajar.

Para ilustrar el uso de las sesiones, veremos la siguiente aplicación web. Para esta aplicación añadiré tres servlets. Cuando se añade un servlet a una aplicación web, no debe colocarse en el paquete por defecto del proyecto. Típicamente, creo un paquete llamado nombredelproyecto.servlets (en este caso, sessions.servlets) y coloco mis servlets allí. En este caso, también he creado un paquete llamado sessions.data, y he añadido la siguiente clase que puede representar a una persona con un **name** y un **title**:

```
public class Person {
    private String name;
    private String title;

    public Person(String name, String title) {
        this.name = name;
        this.title = title;
    }

    public String getName() {
        return name;
    }

    public String getTitle() {
        return title;
    }
}
```

Una clase bien normalilla. Lo siguiente es un servlet:

```
public class CreateServlet extends HttpServlet {
    protected void processRequest(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html; charset=UTF-8");
        try (PrintWriter out = response.getWriter()) {
            String name= request.getParameter("name");
            String title= request.getParameter("title");
            if (name != null && title != null){
                Sesiones.data.Person pers=
```

?

```

        new Sesiones.data.Person(name.trim(), title.trim());
        HttpSession session = request.getSession(true);
        session.setAttribute("Person", pers);
    }
    out.println("<!DOCTYPE html>");
    out.println("<html>");
    out.println("<head>");
    out.println("<title>Servlet CreateServlet</title>");
    out.println("</head>");
    out.println("<body>");
    out.println("<h3>Su objeto de sesion está actualizado</h3>");
    out.println("<h3>Session ID: " + request.getSession().getId() + "</h3>");
    out.println("<a href=\"index.html\">Ir a inicio</a>");
    out.println("</body>");
    out.println("</html>");
}
}

```

El método `processRequest()` comienza determinando los valores de dos campos de formulario de la página que envió una petición a este servlet. Si es posible, y ambos son diferentes de null, se utilizan para crear un objeto `Persona`. A continuación, se añade una referencia al objeto de sesión (hay que importar `javax.servlet.HttpSession`):

Esto significa que el objeto debe crearse si aún no existe. Como paso siguiente, el objeto `Person` se añade a la colección del objeto de sesión con la clave `person`. Por último, se devuelve una respuesta, que incluye el ID del objeto de sesión.

El código de la página de inicio es el siguiente:

```

<!DOCTYPE html>
<!--
Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
Click nbfs://nbhost/SystemFileSystem/Templates/JSP_Servlet/Html.html to edit this template
-->
<html>
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>ServletTest</title>
    </head>
    <body>
        <h3>Start page</h3>
        <form method="post" action="CreateServlet">
            <table>
                <tr>
                    <td>Enter name</td>
                    <td><input type="text" name="name" id="name" /></td>
                </tr>
                <tr>
                    <td>Enter job title<input type="text" name="title" /></td>
                    <td><input type="text" name="title" /></td>
                </tr>
                <tr>
                    <td colspan="2"><input type="submit" value="OK" /></td>
                </tr>
            </table>
        </form>
    </body>
</html>

```

?

```

<ol>
    <li><a href="NameServlet">Muestra name</a></li>
    <li><a href="TitleServlet">Muestra title</a></li>
</ol>
</body>
</html>

```

Si ejecutamos la app, veremos la siguiente página

Start page
 Enter name _____
 Enter job title _____
 Botón OK
 Muestra name (enlace)
 Muestra title (enlace)

Si guardamos datos y pulsamos Ok

Su objeto de sesión está actualizado
 Session ID= (número hexadecimal)

Indica que el objeto de sesión está actualizado, así como cuál es el ID de sesión. Debe tener en cuenta que se representa como un gran número hexadecimal, que generalmente no se puede utilizar para nada, pero en este caso se puede utilizar para ver Indica que el objeto de sesión está actualizado, así como cuál es el ID de sesión.

Lo siguiente es crear un nuevo servlet que muestra el name del objeto Person que se ha creado:

```

public class NameServlet extends HttpServlet {

    /**
     * Processes requests for both HTTP <code>GET</code> and <code>POST</code>
     * methods.
     *
     * @param request servlet request
     * @param response servlet response
     * @throws ServletException if a servlet-specific error occurs
     * @throws IOException if an I/O error occurs
     */
    protected void processRequest(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        try (PrintWriter out = response.getWriter()) {
            String name = null;
            try{
                name=((Sesiones.data.Person)
                    request.getSession().getAttribute("person")).getName();
            }catch(Exception e){
                name = e.toString();
            }
            out.println("<!DOCTYPE html>");
            out.println("<html>");
            out.println("<head>");
            out.println("<title>Servlet NameServlet</title>");
            out.println("</head>");

```

?

```

        out.println("<body>");
        out.println("<h1>Name</h1>");
        out.println("<h3>" + name+"</h3>");
        out.println("<a href=\"index.html\">Ir a inicio</a>");
        out.println("</body>");
        out.println("</html>");
    }
}

```

No hay mucho que explicar, pero debes fijarte en cómo hacer referencia al objeto de sesión y al objeto que se almacena bajo el nombre person. El resultado es un Object, y por lo tanto es necesario un tipo de cast. Hay que submit pulsando OK en el nombre. Luego volver al inicio y ha guardado un nombre en sesion, que lo obtenemos en el enlace Muestra name

Del mismo modo, se ha creado un servlet llamado TitleServlet, que envía una respuesta con el título del objeto Persona. (este lo dejo de tarea)

```

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/JSP_Servlet/Servlet.java to edit this template
 */
package Sesiones.servlets;

import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
/**
 *
 * @author enrique
 */
public class TitleServlet extends HttpServlet {

    /**
     * Processes requests for both HTTP <code>GET</code> and <code>POST</code>
     * methods.
     *
     * @param request servlet request
     * @param response servlet response
     * @throws ServletException if a servlet-specific error occurs
     * @throws IOException if an I/O error occurs
     */
    protected void processRequest(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        try (PrintWriter out = response.getWriter()) {
            String title = null;
            try{
                title=((Sesiones.data.Person)
                    request.getSession().getAttribute("person")).getTitle();
            }catch(Exception e){
                title = e.toString();
            }
            out.println("<!DOCTYPE html>");
            out.println("<html>");
            out.println("<head>");
            out.println("<title>Servlet TitleServlet</title>");
            out.println("</head>");

```

?

```

        out.println("<body>");
        out.println("<h1>Title</h1>");
        out.println("<h3>" + title + "</h3>");
        out.println("<a href=\"index.html\">Ir a inicio</a>");
        out.println("</body>");
        out.println("</html>");
    }
}

// <editor-fold defaultstate="collapsed" desc="HttpServlet methods. Click on the + sign on the left to edit the code.">
/**
 * Handles the HTTP <code>GET</code> method.
 *
 * @param request servlet request
 * @param response servlet response
 * @throws ServletException if a servlet-specific error occurs
 * @throws IOException if an I/O error occurs
 */
@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    processRequest(request, response);
}

/**
 * Handles the HTTP <code>POST</code> method.
 *
 * @param request servlet request
 * @param response servlet response
 * @throws ServletException if a servlet-specific error occurs
 * @throws IOException if an I/O error occurs
 */
@Override
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    processRequest(request, response);
}

/**
 * Returns a short description of the servlet.
 *
 * @return a String containing servlet description
 */
@Override
public String getServletInfo() {
    return "Short description";
}
</editor-fold>
}

```

```

try (PrintWriter out = response.getWriter()) {
    String name= request.getParameter("name");

```

?

```
String title= request.getParameter("title");
if (name != null && title != null){
    Sesiones.data.Person pers=
        new Sesiones.data.Person(name.trim(),title.trim());
    HttpSession session = request.getSession(true);
    session.setAttribute("Person", pers);
}
```

Última modificación: jueves, 23 de mayo de 2024, 11:27